

Agent Factory

Інструкція користувача

Від першого запуску до безпечної роботи з агентами, веб-інтерфейсом, approvals і recovery

Версія	0.1.0
Дата	12 серпня 2026
Режим за замовчуванням	offline / simulation-safe

Зміст

- 1. Що потрібно
- 2. Встановлення
- 3. Перший запуск і стан
- 4. Веб-інтерфейс
- 5. Проекти, задачі та workflow
- 6. Агенти та провайдери
- 7. Налаштування
- 8. Audit, approvals і Human Control Plane
- 9. GitHub
- 10. Backup і recovery
- 11. Docker
- 12. Troubleshooting і безпека
- 13. Корисні документи

1. Що потрібно

Потрібні Git 2.40+ і Python 3.11+. Docker, GitHub CLI та зовнішні provider CLI — опційні. Для першого запуску достатньо Git і Python: deterministic provider працює офлайн.

2. Встановлення

Windows PowerShell

```
git clone <repository-url> AgentFactory
Set-Location AgentFactory
uv venv .venv
uv pip install --python .venv\Scripts\python.exe -e ".[web,dev]"
.\venv\Scripts\python.exe -m agent_factory --help
```

Якщо `uv` недоступний, створіть venv стандартним `python -m venv .venv` і встановіть пакет через `.\venv\Scripts\python.exe -m pip install -e ".[web,dev]"`.

macOS/Linux

```
git clone <repository-url> AgentFactory
cd AgentFactory
python3.11 -m venv .venv
.venv/bin/python -m pip install -e ".[web,dev]"
.venv/bin/python -m agent_factory --help
```

3. Перший запуск і стан

```
agent-factory env check
agent-factory providers status
agent-factory demo
```

За замовчуванням SQLite state лежить у ``<workspace>/agent-factory/state.db``. Для іншого workspace використовуйте ``--workspace PATH``; для іншої БД — ``AGENT_FACTORY_DB``.

Demo створює generic project/work item, запускає offline `delivery` workflow, створює артефакти й зупиняється на human approval. Зовнішні провайдери та GitHub не викликаються.

4. Веб-інтерфейс

Встановіть web extras і запустіть Local Control Center:

```
python -m pip install -e ".[web]"
agent-factory --workspace . web --open
```

Відкрийте ``http://127.0.0.1:8765/``. API docs: ``http://127.0.0.1:8765/api/docs``; OpenAPI: ``/api/openapi.json``. Інший порт: ``agent-factory --workspace . web --port 8877``.

Перед початком роботи відкрийте пункт **System monitor**. Він автоматично перевіряє цілісність SQLite, актуальність міграцій, доступність провайдерів, наявність увімкненого агента, активність emergency stop і поточний runtime. Статус `READY` означає, що блокерів немає; `DEGRADED` показує список причин, які треба усунути. Той самий readiness-контракт доступний через `GET /api/monitor`.

Інтерфейс також показує ready/active/blocked/failed work, runs, provider health, approvals, reviewers, audit events, settings і GitHub dry-run preview. Усі мутації проходять ті самі application services, що й CLI, і вимагають явного підтвердження. Зупинка — `Ctrl+C`.

Для production API задайте `AGENT\_FACTORY\_API\_TOKEN`; endpoint `/api/control/actions` вимагатиме `Authorization: Bearer <token>` і tenant scope.

5. Проекти, задачі та workflow

У **Work item controls** можна завантажити технічне завдання у UTF-8 Markdown/TXT або Agent Factory JSON. Натисніть **Upload technical specification**: локальний Backlog Steward проаналізує структуру заголовків, покаже попередню декомпозицію на epic/story/task (глибші рівні позначаються як subtasks), а окреме підтвердження імпорту створить work items. Файл не виконує команди й не запускає зовнішні провайдери.

```
agent-factory project init --name "Example Product" --description "Перший проект"
agent-factory work-item create --project-id 1 --title "First capability" --description "Deliver one capability" --acceptance "Criterion one"
agent-factory work-item list --project-id 1
agent-factory workflow run --task-id 1 --workflow delivery --mode simulation
agent-factory approvals list
agent-factory approvals approve 1 --note "Evidence reviewed"
```

Перед імпортом backlog:

```
agent-factory backlog validate --path examples/backlog.json
agent-factory backlog import --path examples/backlog.json --project-id 1
```

Approval є людським рішенням: агент не може сам себе approve, release або прийняти фінальний результат.

6. Агенти та провайдери

Перевірка:

```
agent-factory providers status
agent-factory agents list
```

Провайдери: `deterministic` (офлайн), `codex`, `claude`, `gemini`, `antigravity`, `ollama`, `firecrawl`; `openclaw` — health-only. Healthy executable не означає authenticated access або дозвіл на execution.

Підключення провайдера відбувається в його власному CLI/profile. Наприклад, для Ollama:

```
ollama pull qwen2.5-coder:7b
agent-factory providers request ollama --agent coding-worker-ollama --task-id 1
agent-factory providers gates
agent-factory providers approve 1 --note "One bounded local artifact"
agent-factory providers invoke 1
```

Gate одноразовий і прив'язаний до provider, agent та work item. Timeout, interruption або failure не роблять gate повторно придатним — запросіть новий.

У веб-інтерфейсі agent controls дозволяють enable/disable і заміну provider/model для майбутніх assignments. Історичні artifacts не переприсвоюються.

7. Налаштування

Пакетні defaults лежать у `src/agent\_factory/defaults`. Для override:

```
New-Item -ItemType Directory -Force config | Out-Null
Copy-Item src\agent_factory\defaults\*.json config\
$env:AGENT_FACTORY_CONFIG_DIR = (Resolve-Path config).Path
```

У веб UI дозволені лише `dashboard\_refresh\_seconds` (2–60) і `audit\_page\_size` (10–200). Не зберігайте secrets, provider auth files або state DB у Git.

8. Audit, approvals і Human Control Plane

```
agent-factory audit list --limit 100
agent-factory state check
agent-factory approvals list
```

Human Control Plane підтримує tenant-scoped полі `mission\_owner`, `operations\_owner`, `security\_reviewer`. Доступні approve/reject, pause/resume/cancel, recompose, release, emergency-stop, enable/drain/quarantine/replace і irreversible retire. Кожна дія має actor, role, tenant, target і immutable audit record. `retire` вимагає explicit irreversible confirmation.

9. GitHub

GitHub integration plan-first і dry-run by default. Preview перевіряє repository/backlog path і створює SHA-256 mutation plan. Apply потребує окремого human approval; автоматичний merge, close, delete або final approval не виконується.

10. Backup і recovery

```
agent-factory state backup --to /absolute/path/backup.db
agent-factory state check
agent-factory providers reconcile
agent-factory providers gates
```

Зупиніть процеси перед restore, збережіть пошкоджену БД і WAL/SHM, перевірте backup через `PRAGMA integrity\_check`, відновіть у новий шлях і лише потім запускайте reconciliation. Не редагуйте SQLite вручну.

11. Docker

```
docker compose build
docker compose run --rm agent-factory env check
docker compose run --rm agent-factory demo
```

Контейнер працює non-root, має read-only filesystem, persistent `/data`, обмежений `/tmp`, drops capabilities і не містить provider credentials. Docker profile — simulation-safe; реальні provider CLI запускайте на host або в окремому reviewed image.

12. Troubleshooting і безпека

- ``agent-factory`` не знайдено: використайте ``python -m agent_factory --help`` або executable із ``.venv``.
- Provider missing/unhealthy: перевірте його ``--version``, authentication і ``agent-factory providers status``.
- Windows ``WinError 5``: додайте доступний native executable у reviewed config; не вмикайте shell/elevation.
- GitHub Actions jobs не стартують: перевірте [CI troubleshooting](ci-troubleshooting.md); billing/spending-limit блокер GitHub не є помилкою коду.
- Не публікуйте provider output без review. Secrets не повинні потрапляти у prompts, logs, artifacts або commits.

13. Корисні документи

- ``docs/getting-started.md`` — розширене встановлення.
- ``docs/local-control-center.md`` — web UI та API.
- ``docs/providers.md`` — provider-specific setup.
- ``docs/operations.md`` — backup, restore, reconciliation.
- ``docs/human-control-plane.md`` — human actions і authority.
- ``docs/api-contract.md`` — auth, ETags, idempotency, webhooks, SDK contract.
- ``docs/final-audit-2026-08-12.md`` — фінальний аудит рішення.